Problem 8 — Handwritten Solution Guide

Lossless Data Compression | Questions Q1–Q5

Student group — enter group members

Draft for manual transcription; NOT a handwritten submission

How to use this guide

Important: This is a typeset *study and transcription guide*, not a handwritten artifact. The assignment explicitly requires **physically handwritten solutions**. Reproduce the complete calculations, tables, trees, and execution traces by hand, then check legibility and page order before submission. Confirm your group assignment and deadline with the professor.

Conventions and scope

- A message has $N = 100$ symbols, with alphabet $\Sigma = \{A, B, C, D, E, F\}$ and frequencies $45, 13, 12, 16, 9, 5$, respectively.
- Payload-bit counts in **Q1–Q3 exclude file headers and padding**; the implementation benchmark **includes** packet headers and final-byte padding.
- Theoretical RLE Q1 uses 7 bits per count and **3 bits per symbol** (10 bits/run); the general byte-oriented Python RLE codec uses 7 bits per count and **8 bits per byte** (15 bits/run).
- The presentation and code should use the *same* algorithms, parameters, and test inputs whenever they are directly compared.

Q1 — V1 / V2 / entropy / V3 Huffman

1. Initial data and fixed-length encoding

Symbol	A	B	C	D	E	F	Total
Frequency	45	13	12	16	9	5	**100**
Probability p_i	0.45	0.13	0.12	0.16	0.09	0.05	**1.00**

For an alphabet of size $m = 6$, each fixed-length codeword requires

$$b = \lceil \log_2 6 \rceil = \lceil 2.58496 \rceil = 3 \text{ bits.}$$

Therefore,

$$B_{V1} = Nb = 100 \cdot 3 = \boxed{300 \text{ bits}}.$$

Example symbol assignment: $A = 000$, $B = 001$, $C = 010$, $D = 011$, $E = 100$, $F = 101$.

2. Run-length encoding (RLE)

Each run is stored as $(\text{count}, \text{symbol})$ using $7 + 3 = 10$ bits per run, so

$$B_{V2} = 10r \text{ bits},$$

where r is the number of contiguous runs.

Case (a): all identical letters occur together. With one block for each of six symbols, $r = 6$. Hence $B = 6 \cdot 10 = \boxed{60 \text{ bits}}$.

Case (b): no two adjacent letters are equal. Each of the 100 positions is a run, so $r = 100$. Hence $B = 100 \cdot 10 = \boxed{1000 \text{ bits}}$.

This demonstrates that RLE may either reduce or expand the data. Each count is at most 100, so its 7-bit field is sufficient for Q1. In the 5,000-character benchmark runs, the implementation instead splits runs into chunks of at most 127 characters.

3. Shannon entropy (show every contribution)

$$H(X) = -\sum_{s \in \Sigma} p(s) \log_2 p(s).$$

Symbol	p_i	$-p_i \log_2(p_i)$ (bits/symbol)
A	0.45	0.518401
B	0.13	0.382644
C	0.12	0.367067
D	0.16	0.423017
E	0.09	0.312654
F	0.05	0.216096
Sum	**1.00**	**2.219880**

Thus $\boxed{H(X) = 2.220 \text{ bits/symbol}}$ to three decimals, and

$$NH(X) = 100 \times 2.219879978 = \boxed{221.988 \text{ bits}}.$$

Interpretation: the symbol-by-symbol prefix code must have mean length at least $H(X)$. It cannot achieve fractional total transmitted bits; the entropy bound is theoretical. Metadata and padding are not included in this lower bound.

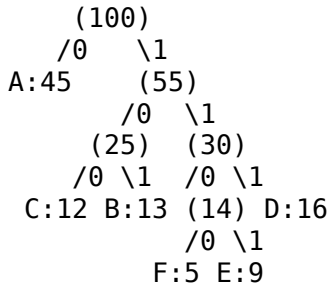
4. Huffman min-heap trace — all five merges

Write the heap state below in **ascending priority order** (this is a *logical priority-queue view*, not the literal array layout of a binary heap). All ties are absent in this example; each merge places the smaller-weight child on edge `0` (left).

Step	Two minima removed	New parent	Remaining min-heap, smallest first
Initial	—	—	F:5, E:9, C:12, B:13, D:16, A:45
1	F:5, E:9	FE:14	C:12, B:13, FE:14, D:16, A:45

Step	Two minima removed	New parent	Remaining min-heap, smallest first
2	C:12, B:13	CB:25	FE:14, D:16, CB:25, A:45
3	FE:14, D:16	FED:30	CB:25, FED:30, A:45
4	CB:25, FED:30	CB-FED:55	A:45, CB-FED:55
5	A:45, CB-FED:55	root:100	root:100

Draw the full tree on paper with `0` on every left edge and 1 on every right edge:



Read the binary path from root to each leaf.

Symbol	Path/codeword	Length l_i	Frequency f_i	$f_i l_i$
A	`0`	1	45	45
B	`101`	3	13	39
C	`100`	3	12	36
D	`111`	3	16	48
E	`1101`	4	9	36
F	`1100`	4	5	20
Total			**100**	**224**

$$B_{V3} = \sum_{s \in \Sigma} f(s)\, l(s) = 45 + 39 + 36 + 48 + 36 + 20 = \boxed{224 \text{ bits}}.$$

Mean length $\bar{L} = 224/100 = 2.240$ bits/symbol. Check: $2.240 \geq 2.219880$; the Huffman payload is $224 - 221.987998 = 2.012002$ bits above the entropy bound.

Why the algorithm is correct: Huffman's greedy-choice property states that two minimum-frequency symbols can be chosen as deepest siblings in some optimal prefix tree. Contracting them into a pseudo-symbol of combined weight reduces the problem by one symbol; repeating this step leads to an optimal tree (induction on alphabet size). Each merge costs $O(\log m)$ heap work and there are $m - 1$ merges; the full procedure requires $O(m \log m)$ time after $O(m)$ heapification.

Q2 — Decode the 13-bit bitstream

Given the *Stage 3* codebook above, decode `0101011001110` left to right. Start at the root, read one bit at a time, and emit a character whenever the path reaches a leaf.

Bits consumed (1-based positions)	Tree walk	Reached symbol	Remaining unconsumed bits
1	`0`	A	`101011001110`
2–4	`1 → 0 → 1 = 101`	B	`011001110`

Bits consumed (1-based positions)	Tree walk	Reached symbol	Remaining unconsumed bits
5	`0`	A	`11001110`
6–9	`1 → 1 → 0 → 0` = `1100`	F	`1110`
10–12	`1 → 1 → 1` = `111`	D	`0`
13	`0`	A	empty

The complete segmentation is `0 | 101 | 0 | 1100 | 111 | 0`, producing **ABAFDA**.

Why unique? The Huffman code is **prefix-free**: none of its codewords is the beginning of a longer codeword. Every leaf completes exactly one symbol; the decoder then resets to the root. Therefore boundaries are determined unambiguously without separators.

Q3 — Canonical Huffman reconstruction

The codeword lengths are $l_A = 1$, $l_B = l_C = l_D = 3$, and $l_E = l_F = 4$. Sort by **(length, alphabetical symbol)**, giving A, B, C, D, E, F.

Start with integer code 0 for the first symbol. For each later symbol apply

$$\text{code}_{\text{next}} = (\text{code}_{\text{previous}} + 1) \ll (l_{\text{next}} - l_{\text{previous}}).$$

Order	Symbol	Length	Arithmetic on integer code	Decimal code	Binary codeword
1	A	1	initialize 0	0	`0`
2	B	3	$(0+1) \ll 2 = 4$	4	`100`
3	C	3	$(4+1) \ll 0 = 5$	5	`101`
4	D	3	$(5+1) \ll 0 = 6$	6	`110`
5	E	4	$(6+1) \ll 1 = 14$	14	`1110`
6	F	4	$(14+1) \ll 0 = 15$	15	`1111`

Prefix-free check: A (`0`) is distinct from every other code starting in `1`. `100`, `101`, and `110` are length-three leaves, and neither `1110` nor `1111` starts with any complete shorter codeword. No codeword prefixes another.

The codeword lengths did not change, so the weighted total is exactly the same as V3:

$$45(1) + 13(3) + 12(3) + 16(3) + 9(4) + 5(4) = \boxed{224 \text{ bits}}.$$

With the alphabet already known, the decoder only needs the length array $[1, 3, 3, 3, 4, 4]$ to reconstruct these canonical codes, rather than the original tree structure. **A real standalone compressed file additionally needs an alphabet/order convention, file length, and packet-format metadata; six lengths are not necessarily the entire physical header.**

V4 format audit (critical distinction): Q3's *theoretical* 224 bits represent the encoded symbol payload only. The statement "six lengths in the header" assumes the six symbols and their fixed order

are already known to the decoder. This project's actual byte-oriented V4 format writes a codebook count plus each (symbol, code_length) pair and also includes a magic identifier, original length, and padding. V3 writes a compact preorder Huffman tree instead. There is **no universal guarantee** that V4's physical header is smaller than V3's: for a complete 256-byte alphabet, the V4 packet can be larger. Compare full packet sizes empirically, without modifying the 224-bit result of the Q3 calculation.

Q4 — LZ77 hand trace: 23-character message

$$S[0..22] = \texttt{ABRACADABRA_ABRACADABRA}, \qquad W = 12, \quad L = 12.$$

The underscore _ in the printed assignment denotes an actual **ASCII space byte (decimal 32, 0x20)**, not an underscore byte (decimal 95, 0x5F). The hand-trace table retains _ as a visual placeholder; the **automated test uses an actual space**. At cursor p, the search buffer is the already processed suffix $S[\max(0, p-12)..p-1]$. A valid triple is $(\text{offset}, \ell, \text{next_char})$ and advances $p' = p + \ell + 1$.

Step	Cursor p	Search buffer	Longest encoded match	Triple $(offset, \ell, next)$	New p'
1	0	(empty)	none	(0,0,A)	1
2	1	A	none (B new)	(0,0,B)	2
3	2	AB	none (R new)	(0,0,R)	3
4	3	ABR	A at source index 0	(3,1,C)	5
5	5	ABRAC	A at source index 3	(2,1,D)	7
6	7	ABRACAD	ABRA at source index 0	(7,4,_)	12
7	12	ABRACADABRA_	ABRACADABR at source index 0	(12,10,A)	23

Why the final match has length 10, not 11: All 11 remaining characters match the previous text, but the assignment's required triple has a mandatory next_char. Reserving the last A as next_char gives $(12, 10, A)$ and consumes exactly $10 + 1 = 11$ characters. This is consistent with the example shown in the problem sheet.

Overlapping references: LZ77 normally permits copying references that overlap the current output; when decoding, copy one byte at a time using the byte located offset positions back, *then* append the next_char. For these 7 tokens, no crossing of the output boundary is necessary, but the implementation supports it.

Check total consumed characters: $1 + 1 + 1 + 2 + 2 + 5 + 11 = \boxed{23}$. Hence the number of emitted triples is $\boxed{7}$.

Independent decoding of Q4 (recommended as a check)

Triple	New decoded message
(0,0,A)	A
(0,0,B)	AB
(0,0,R)	ABR

Triple	New decoded message
(3,1,C)	ABRAC
(2,1,D)	ABRACAD
(7,4,_)	ABRACADABRA_
(12,10,A)	ABRACADABRA_ABRACADABRA

Q5 — Five-stage complexity and capability comparison

Let N be input length, m be alphabet size, W be LZ77 window size, L be lookahead length, and t be number of emitted LZ77 tokens ($t \leq N$). Complexities are for the **encoder**, unless otherwise noted. Exclude encoded output from auxiliary space; the provided V5 batch implementation stores tokens.

Stage	Encoding time	Auxiliary space	Header overhead	Skewed single symbols	Repeated phrases
V1 fixed	$O(N+m)$	$O(m)$	Alphabet, length, code width: $O(m)$	Poor: all symbols equal width	Poor: no dictionary
V2 RLE	$O(N)$	$O(1)$ streaming	Small format metadata	Good only if equal symbols are consecutive	Poor if repeated phrase consists of varied adjacent symbols
V3 tree Huffman	$O(N + m \log m)$	$O(m)$	Explicit tree/symbol structure: $O(m)$ nodes	Very good for skewed frequencies	Cannot exploit phrase repetition beyond byte frequencies
V4 canonical	$O(N + m \log m)$	$O(m)$	Code-length array + symbol mapping: $O(m)$	Same coded payload length as V3	Same phrase limitation as V3
V5 LZ77 + canonical	Naive worst case $O(NWL + t + m' \log m')$	$O(N + W + m')$ for **this batch encoder**	Several codebooks and token metadata; can dominate small inputs	Depends on tokenization, tables, and overhead	Very good when long phrases recur within W bytes

Here m' is the combined number of distinct token symbols across literal, length, and offset alphabets. The implementation uses indexed 3-byte-prefix candidate lists and bounded windows, improving typical search behavior over a naive full window scan; the conservative worst-case search bound remains $O(NWL)$. Because it stores the token list before entropy coding, its auxiliary storage is $O(N + W + m')$, not strictly $O(W)$.

Decoding: V1/V2 are $O(N)$; V3/V4 decoding is $O(B)$ if bit traversal/lookup time is counted over B encoded bits; V5 decoding reconstructs N output bytes and reads Huffman token bits, $O(N+B)$ with canonical decoding tables.

Tradeoffs to explain orally: Fixed-length coding gives a simple baseline. RLE exploits adjacent runs. Huffman codes statistical symbol skew. Canonical Huffman preserves Huffman lengths while reducing

the representation of the dictionary. LZ77 encodes repeated substrings using offsets and lengths, then canonical Huffman encodes the resulting token fields.

Part C — What to show next to the handwritten solution

- Implement **all five** separate `compress` / `decompress` pairs in Python; the project includes `compression_codecs.py` and an executable `.ipynb`.
- Use datasets: A*5000* + *B*5000, weighted random DNA ($p_A = .7$), 50 KiB of actual installed Python standard-library sources, and `os.urandom(10000)`.
- Assert **all 20** dataset/codec byte-for-byte round trips.
- Graph **total packet bits per original byte**, displaying raw data at 8 and empirical zero-order entropy H_0, plus encode/decode throughput in MiB/s. Reproducibility uses SHA-256-pinned inputs, one benchmark CSV, one recorded system environment, a warm-up and five timed repetitions; speed itself is machine-dependent.
- Explain that the compression ratio includes the file header, whereas Q1–Q3 theoretical bit totals do not.
- Cite your *real* GitHub URL in the slides after uploading; the repository URL is intentionally not invented here.

Before submitting — manual pages checklist

1. Write your group identifiers and the exact assigned problem number on the first page.
2. Hand-copy *all five* Q1 heap states and the complete 0/1-labeled Huffman tree.
3. Show each Q1 numeric calculation, the frequency/code table, and the six entropy contributions.
4. Include all six Q2 decoding segments in order, and explain the prefix-free property.
5. Show each Q3 bit shift and the six final canonical codewords.
6. Copy all seven Q4 cursor steps, the final triple count, and one independent decode check.
7. Reproduce the full Q5 complexity comparison with the meaning of N, m, W, L, t.
8. Check page numbering, handwriting legibility, photos/scans, and compliance with any additional rubric.
9. Verify the code, slides and handwritten material all display the same Q1–Q4 values.
10. Confirm actual deadline, group members, GitHub link, and exact submission process before uploading.